Git-Mind: A Self-Healing CI/CD Assistant with Human-in-the-Loop Review

Project Proposal for UCS503P
Submitted to: Miss Nisha Thakur
Thapar Institute of Engineering and Technology

Mohit Srivastav
Roll No: 1024030368

Seerat
Roll No: 1024030361

Shreyas Tiwari
Roll No: 1024030384

Yuvraj Malik
Roll No: 1024030353

August 21, 2026

1 Higher-order goal

. . . secondary framing

This project aims to reduce the engineering time lost to routine, mechanically-fixable continuous-integration failures — the syntax slips, off-by-one errors, and stale logic that interrupt a developer's flow without requiring deep judgement to resolve. While developer productivity is a broad, organisation-wide concern, the immediate engineering objective is to translate *a narrow, well-verified class of automatic fixes* into a measurable, deployable software system, rather than to claim general-purpose autonomous debugging.

2 Time-to-value

. . . primary heuristic

- We validated the highest-risk assumption — whether an LLM can reliably produce a working patch from an error log — with a standalone script before building any surrounding infrastructure, and only proceeded once that assumption held up against a curated benchmark.
- We prioritize fast, falsifiable feedback over an impressive-looking demo: every fix generated by the agent is re-run against the project's own test suite before it is ever committed, so failure is caught before it is shipped rather than discovered later.
- Success in the first iteration is defined narrowly — a working end-to-end loop from a detected bug to a real, human-reviewable pull request on a single-file class of bugs — and is expanded only after that loop is demonstrated, not assumed.

3 Problem Statement

Developers routinely lose time to build and test failures that are mechanically simple to fix but still require a full context switch — reading the log, locating the file, applying the correction, and re-running the pipeline. Common pain points include:

- **Context-switch cost:** a one-line fix can cost several minutes of interrupted focus, disproportionate to the complexity of the bug itself.

- **Low visibility into automation attempts:** when teams do use auto-fix tooling, there is often no transparent record of what was tried, what passed verification, and what was discarded.
- **Unsafe automation:** naively deployed "AI auto-fix" tools risk committing plausible-looking but incorrect patches with no verification gate and no human checkpoint before code ships.
- **Repeated, avoidable toil:** the same narrow classes of bugs (missing imports, off-by-one errors, stale logic) recur across a codebase and are rarely worth a human's full attention individually.

Why this matters now (contextual relevance): As CI pipelines run more frequently and teams ship smaller, faster changes, the cumulative cost of routine failures grows even though each individual failure is trivial to diagnose.

4 Proposed Solution *... Software Proposition*

4.1 Overview

Build an **event-driven, human-in-the-loop** CI-failure remediation system with the following components:

- **Failure ingestion:** a webhook receiver that captures CI failure events (repository, commit, error log) from GitHub.
- **Verification-gated fixer agent:** an LLM-driven worker that proposes a patch, applies it to a working copy, and re-runs the relevant test(s) — a patch is committed *only* if it passes.
- **Pull-request-based delivery:** verified fixes are pushed to a dedicated branch and opened as a normal pull request; the system never merges its own code.
- **Run logging:** every invocation — successful or failed — is recorded with its outcome, failure stage, and retry count, so the system's behaviour is auditable rather than anecdotal.
- **Live dashboard:** a node-graph visualization of repository activity and in-flight AI interventions.

4.2 Core workflow

1. A CI failure is detected (via webhook, or a manually triggered run during the current development phase) and queued with its error log and the implicated file.
2. The fixer agent generates a candidate patch from the error context, retrying up to three times if generation or verification fails, feeding the new failure back into each subsequent attempt.
3. The patch is applied to a fresh branch and the relevant test is executed locally; an unverified patch is discarded and the original file restored, with nothing committed.
4. Only a verified patch is committed, pushed, and opened as a pull request for human review and merge.
5. The dashboard and run log reflect the outcome regardless of whether the attempt succeeded.

4.3 Operational constraints for an educational setting

- The fixer agent's scope is explicitly limited, on current evidence, to single-file, deterministic logic and syntax bugs; cross-file and stateful bug categories are tracked separately and not yet claimed as reliable.
- All git operations are bounded to a configured target repository path, with explicit guards against writing or pushing outside that scope.
- The system never auto-merges; a human reviewer is a mandatory step before any change reaches the main branch.

5 Solution Approach *... Engineering focus*

This is an engineering course project: the contribution is a verified, safety-gated pipeline built on existing LLM and developer tooling, not a new machine-learning model or algorithm.

5.1 Fix generation *... non-research*

Patch generation uses an off-the-shelf LLM API (Gemini, via LangChain) prompted with the error log and file content, with output parsed as a fenced code block rather than structured JSON — a deliberate choice made after empirically observing that JSON-mode output was unreliable for multi-line code compared to a plain code-fence extraction.

5.2 Verification-first architecture *... the core safety mechanism*

Every candidate patch is subjected to the same local test the original failure violated before it is allowed to reach git history. This was validated directly: across an 8-case curated benchmark spanning trivial, reasoning, cross-file, and stateful bug categories, the verification gate correctly distinguished passing patches from a patch that was semantically plausible but violated the expected interface contract, without a human needing to inspect the diff first.

5.3 Event-driven backend

- **Ingestion:** Express server handling GitHub webhooks with signature verification.
- **Queueing:** Redis-backed BullMQ workers, decoupling ingestion from the (slower, LLM-bound) fix-generation step.
- **Persistence:** MongoDB records of every run, shared between the API and worker services via a common schema module rather than direct filesystem coupling between services.
- **Live updates:** Socket.io pushes completed runs to a React Flow–based dashboard.

6 Evaluation Criterion *... measurable and attributable*

Success is defined using metrics captured directly from the run log, not from demo impressions.

6.1 Primary evaluation metric

Verified-fix rate: the percentage of triggered runs, within the currently supported bug categories, that pass local verification and result in an opened pull request.

- Current benchmark evidence: 7 of 8 cases in a curated, cross-category test set passed verification, with the single failure attributable to an unsupported bug category (stateful state-reset behaviour) rather than a verification-gate defect.
- Target for the pilot: $\geq 75\%$ verified-fix rate, measured across repeated runs (not a single pass) per bug category, reported separately by category rather than as one aggregate number.
- Attribution: measured directly from the persisted run log (outcome, attempt count, failure stage) for every invocation.

6.2 Secondary evaluation metrics

- **Time-to-PR:** elapsed time from failure detection to an opened pull request.
- **False-positive rate:** verified patches that pass the local test but are judged incorrect or unsafe on manual review (e.g., a patch that satisfies an assertion by narrowing behaviour rather than fixing it).
- **Category-wise reliability:** pass rate reported separately for trivial, single-file-reasoning, cross-file, and stateful bug categories, since these have shown materially different reliability in testing to date.

6.3 Pilot validation plan *. . . high-level*

- Maintain and extend a curated benchmark repository of synthetic bugs across the categories above, each paired with an explicit assertion rather than a bare exception check.
- Run each benchmark case multiple times (not once) to report an actual pass rate rather than a single-run result, which testing so far has shown can be misleading.
- Compare manual fix time against agent-assisted time on a sample of real, injected CI failures once webhook-triggered automation is in place.

7 Scalability *. . . with foresight*

1. **Scalable (theoretically):** The system is designed to support growth in concurrent repositories and failure volume by:
   - decoupling ingestion from processing via a queue, allowing worker instances to scale independently,
   - sharing schemas between services through a common module rather than cross-service filesystem coupling, so services can be deployed and scaled independently,
   - keeping the API layer stateless.
2. **Operationally deployable (practically):** The system is built entirely on containerizable, commonly hosted components:
   - a managed document database (MongoDB),
   - a managed queue/cache (Redis),
   - standard Node.js services deployable on common container hosting.

8 Engine availability heuristic

A mature set of "engines" (tooling/services) is available to implement this system:

- Express for the webhook receiver and API layer.
- Redis and BullMQ for asynchronous job processing.
- MongoDB for run and repository state persistence.
- GitHub's REST API (via Octokit) and `simple-git` for repository operations.
- An LLM API (Gemini) via LangChain for patch generation.
- React, React Flow, and Socket.io for the live dashboard.

We use these mature components to focus engineering effort on the verification gate, safety guards, and observability of the system, rather than on infrastructure that already exists.

9 Project scope and deliverables *... iteration-friendly*

9.1 Initial deliverable *... already demonstrated*

- A dashboard frontend with a real, API-backed data layer (built first, per course guidance, ahead of the backend).
- A validated fixer-agent core loop: LLM-based patch generation, local test verification before commit, and automatic reversion of unverified patches.
- A real, manually triggered end-to-end run producing an actual GitHub pull request from a detected bug, with the diff manually confirmed to match the expected fix.
- Persistent logging of every run's outcome, retry count, and failure stage, decoupled from any single service's filesystem.

9.2 Subsequent deliverables

- GitHub webhook receiver with signature verification, replacing the current manually triggered flow.
- Redis/BullMQ-based asynchronous processing of incoming failure events.
- Live, socket-driven updates to the dashboard as fixes are attempted and resolved.
- A guard preventing AI-authored fix branches from re-triggering the fixer agent, to avoid self-referential retry loops.
- Expanded benchmark coverage and, where reliability permits, expanded scope into cross-file and stateful bug categories.

10 Risks and mitigations

- **LLM unreliability outside supported bug categories:** mitigated by the verification-before-commit gate (an unverified patch is never committed) and by explicitly scoping claims to categories with demonstrated pass rates rather than claiming general-purpose fixing.

- **Silent bad automation:** mitigated by mandatory human review before merge — the system opens pull requests, it does not merge them.
- **Runaway retries / cost:** capped at three attempts per run, with non-retryable failures (missing configuration, safety-guard violations) short-circuiting immediately rather than exhausting all retries needlessly.
- **Unsafe file or repository access:** mitigated by explicit path-resolution checks that reject any write outside the configured target repository, and by scoping GitHub credentials to a single repository with the minimum required permissions.
- **Self-triggering fix loops:** planned mitigation is to exclude AI-authored branches from re-triggering webhook events, to be implemented alongside the webhook receiver.
- **Evaluation measurement ambiguity:** mitigated by logging outcome, attempt count, and failure stage for every run rather than relying on informal demo impressions.

11 Summary

Git-Mind proposes a verification-gated, human-in-the-loop system that turns routine CI failures into reviewable pull requests rather than autonomous merges. It meets the course criteria by validating its highest-risk assumption before building surrounding infrastructure, defining success through metrics captured directly from a persisted run log rather than demo impressions, and scoping its claims to the bug categories it has actually been shown to handle reliably — expanding that scope only as evidence supports it.